

if-else Conditions & Mini Login System

Table of Contents

1. Introduction to Decision Making
2. What is an if Statement?
3. Syntax of if Statement
4. if-else Statement
5. Nested if Statements
6. elif Ladder
7. Comparison Operators in Conditions
8. Logical Operators in Conditions
9. Real-Life Examples of Conditions
10. Mini Login System
11. Advantages of Conditional Statements
12. Common Errors in Conditions
13. Practice Programs
14. Conclusion

1. Introduction to Decision Making

In programming, decision-making allows a program to choose different actions based on different conditions.

Conditional statements help programs:

- Make decisions
- Validate data
- Control execution flow
- Build interactive applications

Without conditions, programs would always execute the same instructions.

2. What is an if Statement?

The if statement is used to execute a block of code only when a condition is true.

Syntax of if Statement

```
if condition:  
    statement
```

Example

```
age = 18
```

```
if age >= 18:  
    print("Eligible to vote")
```

Output

Eligible to vote

How it Works

- The condition `age >= 18` is checked
 - If the condition is True, the code inside the if block executes
 - If False, the block is skipped
-

3. if-else Statement

The if-else statement is used when we want one block of code to execute if the condition is true and another block if the condition is false.

Syntax

```
if condition:  
    statements  
else:  
    statements
```

Example: Even or Odd

```
num = int(input("Enter a number: "))  
  
if num % 2 == 0:  
    print("Even Number")  
else:  
    print("Odd Number")
```

Explanation

- % gives the remainder
 - If remainder is 0, number is even
 - Otherwise, number is odd
-

4. Nested if Statements

A nested if means placing one if statement inside another if.

Example

```
age = 22
citizen = True
```

```
if age >= 18:
    if citizen:
        print("Eligible to vote")
```

Use Cases

Nested conditions are useful when:

- Multiple checks are required
 - Conditions depend on previous conditions
 - Complex decision-making is needed
-

5. elif Ladder

The elif statement stands for “else if”.

It is used when there are multiple conditions.

Syntax

```
if condition1:
    statement
elif condition2:
    statement
else:
    statement
```

Example: Grade System

```
marks = int(input("Enter marks: "))
```

```
if marks >= 90:  
    print("Grade A")  
elif marks >= 75:  
    print("Grade B")  
elif marks >= 50:  
    print("Grade C")  
else:  
    print("Fail")
```

Explanation

- First condition is checked
 - If false, next condition is checked
 - Process continues until one condition becomes true
-

6. Comparison Operators in Conditions

Comparison operators compare values.

Comparison Operators Table

Operator Meaning

==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

Example

```
a = 10  
b = 20  
  
print(a < b)
```

Output

True

7. Logical Operators in Conditions

Logical operators combine multiple conditions.

Logical Operators Table

Operator Meaning

and	Both conditions must be true
or	At least one condition must be true
not	Reverses the result

Example

age = 25

```
if age > 18 and age < 60:  
    print("Working Age")
```

8. Real-Life Examples of Conditions

Conditional statements are used in:

- ATM machines
 - Login systems
 - Online shopping websites
 - Mobile applications
 - Banking systems
 - Result processing systems
 - AI systems
 - Security verification systems
-

9. Mini Login System

What is a Login System?

A login system checks whether the entered username and password are correct.

It is one of the most common real-world applications of conditional statements.

Mini Login System Program

```
correct_username = "admin"  
correct_password = "1234"
```

```
username = input("Enter username: ")  
password = input("Enter password: ")
```

```
if username == correct_username and password == correct_password:  
    print("Login Successful")  
else:  
    print("Invalid Username or Password")
```

Explanation of Program

Step 1

Store correct username and password.

```
correct_username = "admin"  
correct_password = "1234"
```

Step 2

Take input from user.

```
username = input("Enter username: ")  
password = input("Enter password: ")
```

Step 3

Check both conditions using and.

```
if username == correct_username and password == correct_password:
```

Step 4

Display result.

```
print("Login Successful")
```

or

```
print("Invalid Username or Password")
```

Sample Output

Correct Credentials

Enter username: admin
Enter password: 1234
Login Successful

Wrong Credentials

Enter username: user
Enter password: 1111
Invalid Username or Password

10. Advantages of Conditional Statements

- Helps in decision making
 - Makes programs intelligent
 - Improves control flow
 - Enables validation systems
 - Useful in automation
 - Required for real-world applications
-

11. Common Errors in Conditions

1. Using = instead of ==

Wrong:

if a = 10:

Correct:

if a == 10:

2. Indentation Errors

Python uses indentation to define blocks.

Wrong indentation causes errors.

3. Forgetting Colon :

Wrong:

```
if age > 18
```

Correct:

```
if age > 18:
```

12. Practice Programs

Program 1: Check Positive or Negative

```
num = int(input("Enter number: "))
```

```
if num >= 0:
    print("Positive")
else:
    print("Negative")
```

Program 2: Largest of Two Numbers

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
```

```
if a > b:
    print("A is greater")
else:
    print("B is greater")
```

Program 3: Password Checker

```
password = input("Enter password: ")
```

```
if password == "python123":
    print("Access Granted")
else:
    print("Access Denied")
```

13. Conclusion

Conditional statements such as if, if-else, and elif are essential programming concepts used to make decisions in programs. They allow developers to create intelligent and interactive systems.

The Mini Login System demonstrates how conditions are used in real-world applications for authentication and security.

Mastering conditional statements is important because they form the foundation for advanced programming concepts like loops, functions, databases, and Artificial Intelligence systems.